Java8 Features

Java 8, released in March 2014, introduced several new features that had a significant impact on Java programming. Here's a detailed explanation of some key features:

Lambda Expressions: This is one of the most notable features of Java 8. Lambda expressions essentially allow you to write instances of single-method interfaces (functional interfaces) more concisely.

Example:

```
List<String> names = Arrays.asList("John", "Jane", "Adam", "Tom");
Collections.sort(names, (String a, String b) -> {
 return b.compareTo(a);
});
```

Here, a lambda expression is used to define the comparison method for sorting.

In this code, we have a list of names, and we want to sort them in reverse order (from 'Z' to 'A'). We use a lambda expression (String a, String b) -> { return b.compareTo(a); } to define the comparison logic for the Collections.sort() method. This lambda expression takes two String parameters a and b, compares them using compareTo(), and returns the result. It's a concise way to define custom sorting behavior.

Stream API: The Stream API is used to process collections of objects in a functional manner. It provides a new abstraction called Stream that lets you process data in a declarative way.

Example:

```
List<String> names = Arrays.asList("John", "Jane", "Adam", "Tom");
names.stream().filter(name ->
name.startsWith("J")).forEach(System.out::println);
```

This code filters a list of names and prints those starting with "J".

Here, we have a list of names, and we want to filter and print names that start with the letter 'J'. We use the Stream API to achieve this. names.stream() converts the list into a stream, filter(name -> name.startsWith("J")) filters the names based on the condition, and forEach(System.out::println) prints each filtered name. It's a more declarative and functional approach to processing data.

Method References: Method references are a shorthand notation of a lambda expression to call a method.

Example:

```
List<String> names = Arrays.asList("John", "Jane", "Adam", "Tom");
names.forEach(System.out::println);
```

Here, System.out::println is a method reference.

In this code, we have a list of names, and we want to print each name. Instead of using a lambda expression, we use a method reference System.out::println, which is a shorthand notation for calling the println method of the System.out object. It makes the code more concise and readable.

Default Methods: Java 8 allows interfaces to have default and static methods. The default method can be provided to an interface without affecting implementing classes as it includes an implementation.

Example:

```
interface Vehicle {
 default void print() {
 System.out.println("I am a vehicle!");
 }
}
```

This code defines an interface Vehicle with a default method print(). Default methods have a default implementation in the interface itself. Any class that implements this interface can choose to override this method or use the default implementation. It allows adding new methods to interfaces without breaking existing implementations.

Optional Class: Optional is a new container type that wraps a value and offers a method to deal with the case when the value is null, thus helping in avoiding NullPointerException.

Example:

```
Optional<String> optional = Optional.of("hello");
if (optional.isPresent()) {
 System.out.println(optional.get());
}
```

Here, we create an Optional object that wraps a value ("hello"). optional.isPresent() checks if the value is present, and if it is, we use optional.get() to retrieve and print the value. Using Optional helps avoid NullPointerException when working with potentially null values.

New Date-Time API: Inspired by Joda-Time, the new Date-Time API in Java 8 is immutable and supports many operations for date and time.

Example:

```
LocalDate localDate = LocalDate.now();
System.out.println(localDate);
```

This code demonstrates the use of the new Date-Time API. We create a LocalDate object representing the current date using LocalDate.now() and then print it. The new Date-Time API provides a more comprehensive and immutable way to work with dates and times.